# Practical Lab: Uncertainty Quantification and Decision-Making in AI

Softmax probability distribution, Calibration, Conformal Prediction, Deep Ensembles and Utility

**Central question**

**Can we build an AI classifier that knows when it is likely to be wrong, and can use this information to make better decisions?**

## Learning objectives

At the end of this practical lab, you should be able to:

- Train a neural network classifier and interpret its softmax probabilities;
- Distinguish prediction accuracy from probability reliability;
- Evaluate calibration using reliability diagrams and Expected Calibration Error (ECE);
- Calibrate a neural network using temperature scaling;
- Understand conformal prediction (CP) and construct CP sets with a coverage guarantee;
- Understand the concept of Deep Ensemble and the ways to implement it;
- Distinguish uncertainty quantification from decision-making;
- Use a utility matrix to compute expected utility and compare prediction with abstention;

During the whole practical lab, we will use the Fashion-MNIST dataset which is described in the following section:

## Experimental setting: Fashion-MNIST

Fashion-MNIST contains 70,000 grayscale images of size $28 \times 28$ belonging to ten clothing classes. It is deliberately simple enough for a short laboratory while still containing ambiguous classes such as Shirt, T-shirt/top, Coat and Pullover.

## Part I – Baseline neural classifier

Let $x$ denote an image and $Y \in \{1, \dots, K\}$ its class. A neural network produces logits

$$z_1(x), \dots, z_K(x).$$

The softmax transformation gives:

$$p(Y = k \mid x) = \frac{\exp(z_k(x))}{\sum_{j=1}^{K} \exp(z_j(x))}.$$

Usually, the predicted class is:

$$\hat{y}(x) = \arg\max_k p(Y = k \mid x),$$

and the corresponding confidence is:

$$c(x) = p(Y = \hat{y}(x) \mid x).$$

**Concept: a probability is not automatically a calibrated probability**

If a classifier assigns confidence 0.9 to a large collection of examples, calibration means that approximately 90% of those predictions should be correct. A neural network can therefore have high accuracy while producing overconfident probabilities.

**Task 1**

Load the fashion MNIST dataset. We will use the standard split:

- training set: model fitting;
- calibration/validation set: temperature scaling, Conformal Prediction or any other tuning;
- test set: final evaluation only.

**Important methodological rule**

Do not tune a method on the test set. In particular, the test set must not be used to choose the temperature, the conformal quantile, the rejection threshold, the utility parameters or any other parameters/hyperparameters.

**Task 2**

Train the baseline CNN (use a simple structure). Report:

1. Test accuracy;
2. Confusion matrix;
3. Examples for which the model is correct with high confidence;
4. Examples for which the model is wrong with high confidence;
5. The complete predictive probability vector for several ambiguous examples.

# Part II – Calibration

## 1. Calibration evaluation

One way to evaluate the calibration of a model is to use the reliabilty diagram and the Expected Calibration Error (ECE) as follows.
For each test example $i$ define:

$$c_i = \max_k p_i(k), \qquad r_i = 1(\hat{y}_i = Y_i).$$

Partition $[0, 1]$ into $m$ bins $B_{j,j\in[\![1,m]\!]}$. Each test example $i \in [\![1, N]\!]$ is assigned to bin $B_j$ if its confidence score $c_i$ falls within the corresponding interval. Define:

$$\text{conf}(B_j) = \frac{1}{|B_j|} \sum_{i \in B_j} c_i$$

and

$$\mathrm{acc}(B_j) = \frac{1}{|B_j|} \sum_{i \in B_j} r_i.$$

A perfectly calibrated classifier satisfies approximately:

$$\mathrm{acc}(B_j) = \mathrm{conf}(B_j).$$

The empirical ECE is:

$$ECE = \sum_j \frac{|B_j|}{N} \left|\mathrm{acc}(B_j) - \mathrm{conf}(B_j)\right|.$$

ECE is useful but should not be treated as an absolute statistical truth: it depends on binning and finite-sample effects.
The reliability diagram illustrates calibration on a bin-by-bin basis by comparing the observed accuracy with the predicted confidence in each confidence bin (see Figure1).

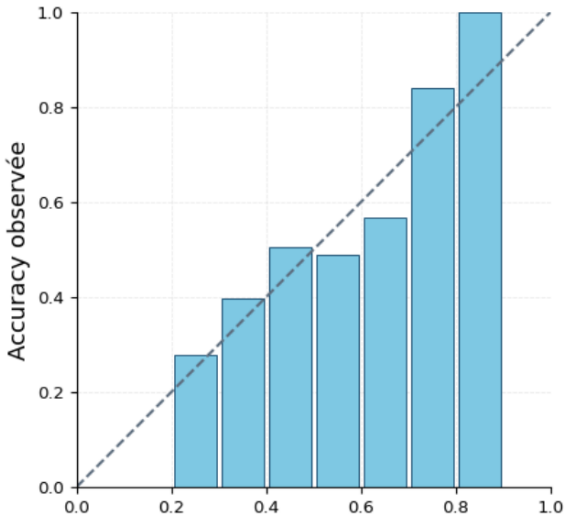



Figure 1: Reliability diagram.

**Task 3**

Produce a reliability diagram and compute accuracy and ECE. Answer:

1. Is the model overconfident or underconfident?
2. Can a model have excellent accuracy and poor calibration?
3. Interpret the reliability diagram.
4. What is the link between ECE and the reliability diagram?

## 2. Calibrate the model: Temperature scaling

One way to improve the calibration of our model is to use the simple temperature scaling method which modifies logits according to:

$$p_T(Y = k \mid x) = \frac{\exp(z_k(x)/T)}{\sum_j \exp(z_j(x)/T)}.$$

Only one scalar parameter $T > 0$ is learned, normally by minimizing Negative Log-Likelihood (NLL) on a validation/calibration set.

If $T > 1$, predictions become softer; if $T < 1$, they become sharper.

**What temperature scaling does – and does not do**

Temperature scaling changes the confidence values without changing the predicted class:

$$\arg\max_k z_k = \arg\max_k \frac{z_k}{T}.$$

It can therefore improve probability reliability without improving classification accuracy.

**Task 4**

Learn $T$ on the calibration set. Compare on the test set before/after temperature scaling:

- Accuracy;
- NLL;
- ECE;
- Reliability diagram.

Explain why the accuracy should remain unchanged.

## Part III – Conformal Prediction

Conformal Prediction constructs a set of plausible labels:

$$\Gamma(x) \subseteq \mathcal{Y}.$$

The split conformal prediction procedure works as follow:

- On the calibration set of size $n$, we compute the non-conformity scores $s_1, \ldots, s_n$. When using a neural network for a classification task, we generally use the score of one minus the softmax output of the true class $Y$:
$$s(x, Y) = 1 - p(Y \mid x).$$
- We compute an appropriate upper empirical quantile $q_{1-\alpha}$ (the $\frac{\lceil (1-\alpha)(n+1) \rceil}{n}$ quantile) on the non-conformity scores. $1 - \alpha$ is called the confidence level.
- Using the non-conformity scores defined above, the prediction set is:
$$\Gamma(x) = \{k : 1 - p(k \mid x) \leq q_{1-\alpha}\}.$$

When the calibration and the test sets are exchangeable, we have:

$$\mathbb{P}(Y_{test} \in \Gamma(X_{test})) \geq 1 - \alpha$$

**A key difference**

A neural network says:

$$p(Y = \text{Shirt} \mid x) = 0.62.$$

Conformal Prediction says:

$$\Gamma(x) = \{\text{Shirt}, \text{T-shirt}\}$$

with a true probability of success.

Important metrics are:

$$\text{Coverage} = \frac{1}{N}\sum_i 1(y_i \in \Gamma(x_i))$$

and

$$\text{Average Set Size} = \frac{1}{N}\sum_i |\Gamma(x_i)|.$$

**Task 5**

For $\alpha = 0.1$ and $\alpha = 0.05$:

1. Construct prediction sets;
2. Measure coverage;
3. Measure average set size;
4. Visualize examples with singleton and multi-label prediction sets.

Explain the coverage–set-size trade-off.

## Part IV – Deep Ensemble

Train $M$ neural networks:

$$p_m(Y = k \mid x), \qquad m = 1, \ldots, M.$$

Then, to aggregate the models, we can use *soft voting* which consists in:

$$\bar{p}(Y = k \mid x) = \frac{1}{M}\sum_{m=1}^{M} p_m(Y = k \mid x).$$

and then:

$$\hat{y}(x) = \arg\max_k \bar{p}(Y = k \mid x),$$

There are various methods to create $M$ different neural networks (we call this diversity):

- Same overall structure but different weight initialization
- Same overall structure but different shuffle during batch creation
- Different structures
- Monte Carlo Dropout where neurons are randomly turned-off during testing
- Monte Carlo DropConnect where connexions between neurons are randomly turned-off during testing
- ...

Note that Monte Carlo Dropout and Monte Carlo DropConnect are not Deep Ensemble methods in the strict sens as for each method only one model is trained with changes occurring during the test phase.
To quantify the uncertainty in deep ensemble, we generally use various entropies:

- The total predictive entropy:

$$H_{total} = -\sum_k \bar{p}(Y = k \mid x)\log(\bar{p}(Y = k \mid x)).$$

- The aleatoric uncertainty:

$$H_{aleatoric} = \frac{1}{M} \sum_{m=1}^{M} \sum_{k} p_m(Y = k \mid x) \log(p_m(Y = k \mid x))$$

- The epistemic entropy (also call disagreement):

$$H_{epistemic} = H_{total} - H_{aleatoric}.$$

  The first term measures total predictive uncertainty, while the second term measures the average uncertainty within the individual models. Their difference is associated with model disagreement.

**Interpretation**

This decomposition is a useful pedagogical approximation to the distinction between aleatoric and epistemic uncertainty in this setting. It should not be interpreted as a universal, exact separation of all real-world uncertainty sources.

**Task 6**

For $M = 5$ networks:

1. Compute the ensemble predictive distribution using, at least, two diversity methods;
2. Compute total predictive entropy;
3. Compute the disagreement indicator above;
4. Compare individual predictions with the ensemble;
5. Visualize examples with high and low disagreement.

Discuss:

- What happens when all networks agree?
- What happens when networks disagree?
- Can the ensemble be uncertain even when the average prediction is relatively confident?

## Part V – From uncertainty to decisions

Prediction and decision are not the same problem. One possible method to decide which action $a$ has to be taken is, given the softmax probability distribution, to take $a = \hat{y}(x) = \arg\max_k p(Y = k \mid x)$ (maximum-probability decision method). However, this method is pretty simple and doesn't consider the potential consequences of an action.
In many decision task, especially in sensitive fields, there are decision-makers (DMs) who will use the predictions to decide. In this case, one possible method is that DMs define an utility function $U(a, y)$ which represents the utility of taking action $a$ when the unknown true class is $y$.

Another possible decision method is then the maximum expected utility decision method which is defined as follows:

- Given a predictive distribution $p(y \mid x)$, the expected utility of action $a$ is:

$$EU(a \mid x) = \sum_{y} U(a, y) p(y \mid x).$$

- The optimal decision is:

$$a^*(x) = \arg\max_a EU(a \mid x).$$

For example, with a zero-one utility: $U(a, y) = \begin{cases} 1, & a = y \\ 0, & a \neq y \end{cases}$, the maximum expected utility decision method reduces to maximum-probability decision method.
But real decisions may assign different costs to different errors. For example, confusing Shirt and T-shirt may be less costly than confusing Shirt and Ankle boot. This can be represented by a non-uniform utility matrix.

**Task 7**

Define a utility matrix with:

- positive utility for correct predictions;
- negative utility for incorrect predictions;
- different penalties for semantically similar and very different classes.

Compare:

1. maximum-probability decision method;
2. maximum expected utility decision method;

Determine which strategy maximizes average utility on the complete test set.

Suppose the system may refuse to make a prediction. Let

$$a \in \{1, \ldots, K, \text{reject}\}.$$

**Task 8**

1. Define a rejection utility $U(\text{reject}, y) = u_R$.
2. Using the maximum expected utility decision method, compute the average utility on the complete test set.

Another decision method is the confidence-based policy:

$$a(x) = \begin{cases} \hat{y}(x), & c(x) \geq \tau, \\ \text{reject}, & c(x) < \tau. \end{cases}$$

with $c$ being the corresponding confidence of prediction $\hat{y}$.

**Task 9**

Study the effect of $\tau \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$ reporting:

- rejection rate;
- accuracy on accepted examples;
- average utility on the complete test set.

**Important message**

There is no universally optimal confidence threshold. The optimal threshold depends on what an incorrect decision costs relative to abstaining.